



Introduction to Linux OS

By Mohamed Gamal



Listing Processes and Displaying Information

Process Types:

Running	The process is either executing on CPU or waiting to run.
Sleeping	The process is waiting for an action to be done in order to be running.
Stopped	The process has been Stopped (suspended).
Zombie	A process which is killed but still shows its entry in the process status or the process table is called a zombie process.

- A **process** is a program which is being executed.
- Any process may create a child process.
- All processes are descendants of the first system process, which is system.
- A process can run in the background or foreground.

```
$ echo $$      # See the PID of your current shell process
$ exit
$ echo $$      # Different PID
```

The **ps** (process status) command in Linux is used to list processes and display information about them. It provides various options to customize the output, depending on what information you want. The **ps** command uses **/proc** directory (`$ ls -lh /proc`) to gather information about running processes.

`$ ps [options]`

Option	Description
<code>-e</code>	Everything, all processes.
<code>-f</code>	Full format listing.
<code>-u <username></code>	Display processes running as username.
<code>-p <PID></code>	Display process information for a PID .
<code>-l</code>	Display detailed information (long format).
<code>aux</code>	Display a comprehensive list of all processes currently running.
...	...

Other Similar Command

<code>\$ pstree</code>	Display running processes in a tree format.
------------------------	---



```

mg@192:~$ ps -el | head -n 10
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0         1         0  0   80   0 - 43505 -    ?           00:00:01 systemd
1 S   0         2         0  0   80   0 -    0 -    ?           00:00:00 kthreadd
1 S   0         3         2  0   80   0 -    0 -    ?           00:00:00 pool_workqueue
1 I   0         4         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-rcu_g
1 I   0         5         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-rcu_p
1 I   0         6         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-slub
1 I   0         7         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-netns
1 I   0         9         2  0   60 -20 -    0 -    ?           00:00:00 kworker/0:0H-events_highpri
1 I   0        10         2  0   80   0 -    0 -    ?           00:00:00 kworker/u512:0-events_unbound

```

- » **F (Flags):** indicates process privileges (e.g., 0 = normal, 4 = superuser).
- » **S (State):** R = Running, S = Sleeping, D = Uninterruptible sleep, Z = Zombie, T = Stopped.
- » **UID (User ID):** the ID of the process owner.
- » **PID (Process ID):** the unique identifier of the process.
- » **PPID (Parent Process ID):** the process that started this one.
- » **C (CPU Usage):** Percentage of CPU usage.
- » **PRI (Priority):** Process priority (lower values mean higher priority).
- » **NI (Nice value):** affects scheduling priority, 0 = default.
- » **ADDR (Address):** Memory address of the process (usually - on modern systems).
- » **SZ (Size):** Process size in memory (in pages).
- » **WCHAN (Waiting Channel):** function where the process is sleeping.
- » **TTY (Teletypewriter):** Terminal associated with the process (e.g., pts/0 for an SSH or terminal session).
- » **TIME:** Total time consumed by the process.
- » **CMD:** Command used to launch the process.

Examples
<pre> \$ ps \$ ps -ef \$ ps -ef head -n 5 \$ ps -fu MG \$ ps -p 10 \$ ps aux head \$ ps aux grep -i vim \$ ls /proc/ \$ ps aux grep 813 # which is shown in /proc </pre>

--- Finding Process IDs ---

Command	Description	Example
<pre> \$ pidof <process name> \$ pgrep <process name> </pre>	Find the PID of a running program.	<pre> \$ pidof vim \$ pgrep vim </pre>



Here're other commands that allow you to view running processes as well, but for real-time monitoring.

--- Real-Time Process Monitoring ---

Option	Description
top	Interactive <u>main processes</u> viewer, for child processes use ps. » Type 1 to display all CPU cores' activity individually. » Type s to change the refresh rate (delay between updates). » Type h for help. » Type k to kill a process by entering its PID. » Type r to renice a process to change its priority (-20 to 19, where lower is higher priority). » Press q to quit. » Type z or Z to toggle color display for better readability. » Type n to change the number of processes shown. » Type O to filter processes based on a specified field (e.g., COMMAND=NAME). » Type Shift+P to sort by CPU usage. » Type Shift+T to sort by process time (CPU time used). » Type Shift+M to by memory usage.
uptime	Display for how long the system has been running.

\$ top

```

MG@centos9:~$ top
top - 20:42:56 up 1:13, 2 users, load average: 2.91, 1.90, 0.87
Tasks: 234 total, 4 running, 230 sleeping, 0 stopped, 0 zombie
%Cpu(s): 68.5 us, 30.9 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.6 hi, 0.0 si, 0.0 st
MiB Mem : 3626.1 total, 1434.3 free, 1399.7 used, 1065.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2226.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
 6210 MG        20   0   779384   59072   44284 R   29.8   1.6   1:40.84 gnome-terminal-
 7458 MG        20   0   222520    3328    3072 R   29.8   0.1   1:23.79 sh
 4525 MG        20   0   3680408 427580 136752 R   28.3  11.5   2:28.72 gnome-shell
 7244 root        20   0         0         0      0 I    9.0   0.0   0:16.83 kworker/u512:3-events_unbound
 6886 root        20   0         0         0      0 I    1.4   0.0   0:15.98 kworker/u512:1-events_unbound
    1 root        20   0   108304   16716   10816 S    0.0   0.5   0:42.07 systemd
    2 root        20   0         0         0      0 S    0.0   0.0   0:00.01 kthreadd
    3 root        20   0         0         0      0 S    0.0   0.0   0:00.00 pool_workqueue_
    4 root        0 -20         0         0      0 I    0.0   0.0   0:00.00 kworker/R-rcu_g
    5 root        0 -20         0         0      0 I    0.0   0.0   0:00.00 kworker/R-rcu_p
    6 root        0 -20         0         0      0 I    0.0   0.0   0:00.00 kworker/R-slub
    7 root        0 -20         0         0      0 I    0.0   0.0   0:00.00 kworker/R-netns
    9 root        0 -20         0         0      0 I    0.0   0.0   0:00.00 kworker/0:0H-kblockd
   11 root        0 -20         0         0      0 I    0.0   0.0   0:00.00 kworker/R-mm_pe
   13 root        20   0         0         0      0 I    0.0   0.0   0:00.00 rcu_tasks_kthre
   14 root        20   0         0         0      0 I    0.0   0.0   0:00.00 rcu_tasks_rude
   15 root        20   0         0         0      0 I    0.0   0.0   0:00.00 rcu_tasks_trace
   16 root        20   0         0         0      0 S    0.0   0.0   0:00.09 ksoftirqd/0
   17 root        20   0         0         0      0 I    0.0   0.0   0:00.23 rcu_preempt
   18 root        rt    0         0         0      0 S    0.0   0.0   0:00.00 migration/0
   19 root       -51   0         0         0      0 S    0.0   0.0   0:00.00 idle_inject/0
   21 root        20   0         0         0      0 S    0.0   0.0   0:00.00 cpuhp/0
   23 root        20   0         0         0      0 S    0.0   0.0   0:00.00 kdevtmpfs

```



--- Understanding `Top` Attributes ---

PID	The ID of the running process.
USER	The process owner.
PR	The priority of the process.
NI	The nice value of the process.
VIRT	The virtual memory size used by the process.
RES	The resident memory size used by the process (physical memory).
SHR	The amount of shared memory used by the process.
S	The state of the process. » R: Running. » S: Sleeping. » D: Uninterruptible sleep. » Z: Zombie. » T: Stopped or traced. » I: Idle
%CPU	The CPU usage percentage by the process.
%MEM	The memory usage percentage by the process.
TIME+	The total CPU time the process has used since it started.
COMMAND	The name or command that started the process.



Filtering processes by their name (e.g., firefox) by pressing 'o'.

```
MG@centos9:~ - top
top - 20:43:38 up 1:13, 2 users, load average: 2.82, 2.01, 0.95
Tasks: 234 total, 5 running, 229 sleeping, 0 stopped, 0 zombie
%Cpu(s): 68.7 us, 31.0 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.3 hi, 0.0 si, 0.0 st
MiB Mem : 3626.1 total, 1428.6 free, 1405.3 used, 1065.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2220.7 avail Mem
add filter #1 (ignoring case) as: [!]FLD?VAL COMMAND=firefox
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
6210 MG 20 0 779384 59072 44284 R 29.8 1.6 1:53.50 gnome-terminal-
7458 MG 20 0 222520 3328 3072 R 29.8 0.1 1:36.45 sh
4525 MG 20 0 3680408 427580 136752 R 28.1 11.5 2:40.68 gnome-shell
7244 root 20 0 0 0 0 I 5.3 0.0 0:18.85 kworker/u512:3-events unbound
6886 root 20 0 0 0 0 I 4.6 0.0 0:18.39 kworker/u512:1-writeback
5127 MG 20 0 394456 44332 33548 R 0.7 1.2 0:05.32 vmttoolsd
930 root 20 0 383680 8960 7552 S 0.3 0.2 0:05.17 vmttoolsd
7474 MG 20 0 225912 4224 3328 R 0.3 0.1 0:00.10 top
1 root 20 0 108304 16716 10816 S 0.0 0.5 0:42.07 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:00.01 kthreadd
3 root 20 0 0 0 0 S 0.0 0.0 0:00.00 pool_workqueue_
4 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_g
5 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_p
6 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-slub
7 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-netns
9 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/0:0H-kblockd
11 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-mm_pe
13 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_kthre
14 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_rude
15 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_trace
16 root 20 0 0 0 0 S 0.0 0.0 0:00.09 ksoftirqd/0
17 root 20 0 0 0 0 I 0.0 0.0 0:00.23 rcu_preempt
18 root rt 0 0 0 0 S 0.0 0.0 0:00.00 migration/0
```

Toggle color display for better readability by pressing 'z'.

```
MG@centos9:~ - top
top - 21:01:30 up 1:31, 2 users, load average: 2.39, 2.26, 1.94
Tasks: 235 total, 1 running, 234 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.3 us, 0.3 sy, 0.0 ni, 97.7 id, 0.0 wa, 0.3 hi, 0.3 si, 0.0 st
MiB Mem : 3626.1 total, 1397.2 free, 1437.0 used, 1065.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2189.0 avail Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
4525 MG 20 0 3648564 398892 136368 S 1.3 10.7 5:04.93 gnome-shell
7565 MG 20 0 225940 4096 3328 R 0.3 0.1 0:00.09 top
1 root 20 0 108304 16716 10816 S 0.0 0.5 0:01.46 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:00.01 kthreadd
3 root 20 0 0 0 0 S 0.0 0.0 0:00.00 pool_workqueue_
4 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_g
5 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_p
6 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-slub_
7 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-netns
9 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/0:0H-kblockd
11 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-mm_pe
13 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_kthre
14 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_rude_
15 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_trace
16 root 20 0 0 0 0 S 0.0 0.0 0:00.09 ksoftirqd/0
17 root 20 0 0 0 0 I 0.0 0.0 0:00.28 rcu_preempt
18 root rt 0 0 0 0 S 0.0 0.0 0:00.00 migration/0
19 root -51 0 0 0 0 S 0.0 0.0 0:00.00 idle_inject/0
21 root 20 0 0 0 0 S 0.0 0.0 0:00.00 cpuhp/0
23 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kdevtmpfs
24 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-inet_
25 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kauditd
27 root 20 0 0 0 0 S 0.0 0.0 0:00.00 khungtaskd
```

...



Running Processes in the Foreground and Background

Up until this point all the commands we have been executing have been running in the foreground. When a command, process, or program is running in the foreground the shell prompt will not be displayed until that process exits.

Why run processes in the background?

- For long running programs it can be convenient to send them to the background.
- Processes that are backgrounded still execute and perform their task, however, they do not block you from entering further commands at the shell prompt.

Command	Description	Example
\$ <command>	Start command in the <u>foreground</u> .	\$ ls -lah
\$ <command> &	Start command in the <u>background</u> .	\$ sleep 1000s &
CTRL + C	<u>Kill</u> the running <u>foreground</u> process.	–
CTRL + Z	<u>Suspend</u> the <u>foreground</u> process.	–
\$ bg [%num]	Background a suspended process.	\$ bg %1
\$ fg [%num]	Foreground a background process.	\$ fg %1
\$ kill [%num]	Kill a process by job number or PID.	\$ kill %1
\$ jobs -l [%num]	List n jobs. » Plus sign (+) : the current job. » Minus sign (-) : the previous job.	\$ jobs -l \$ jobs -l %2

```
MG@centos9:~$ sleep 10000
^Z
[1]+  Stopped                  sleep 10000

MG@centos9 ~]$ bg %1
[1]+  sleep 10000 &

MG@centos9 ~]$ fg %1
sleep 10000
^C
MG@centos9 ~]$
```

```
MG@centos9 ~]$ sleep 1000 &
[1] 7832

MG@centos9 ~]$ kill %1
[1]+  Terminated              sleep 1000

MG@centos9 ~]$ sleep 100000 &
[1] 7846

MG@centos9 ~]$ sleep 200000 &
[2] 7851

MG@centos9 ~]$ jobs
[1]-  Running                  sleep 100000 &
[2]+  Running                  sleep 200000 &
```



Killing Processes

Killing processes in Linux involves terminating processes by sending signals, typically using commands like `kill`, `killall`, or `killall`.

Command	Description
<code>CTRL + C</code>	Kill the running <u>foreground</u> process.
<code>\$ kill -[signal] <PID></code>	Send a signal to a process by <u>PID</u> .
<code>\$ kill -l</code>	Display a list of signals.
<code>\$ man 7 signal</code>	Display the manual for all signals.
<code>\$ pkill <process name></code>	Send a signal to process by <u>name</u> .
<code>\$ killall <process name></code>	Sends a signal to all processes with a specific name.

- » **SIGINT (2)**: Interrupt signal, sent when you press **CTRL+C** in the terminal.
- » **SIGTSTP (20)**: Stop/Suspend a process from the terminal (**Ctrl + Z**).
- » **SIGTERM (15)**: Politely asks a process to terminate gracefully, this is the default signal.
- » **SIGKILL (9)**: Forces termination if a process does not respond.
- » **SIGSTOP (19)**: Stops (pauses) a process, cannot be ignored.
- » **SIGCONT (18)**: Resumes a stopped process.

Running all the following is equivalent which is the default behavior.

```
$ kill <PID>
$ kill -15 <PID>
$ kill -TERM <PID>
$ kill -SIGTERM <PID>
```

```

[MG@centos9 ~]$ sudo tail -f /var/log/messages # Start a dummy process
Dec 5 22:15:18 centos9 systemd[1]: dnf-makecache.service: Deactivated successfully.
Dec 5 22:15:18 centos9 systemd[1]: Finished dnf makecache.
Dec 5 22:25:18 centos9 cupsd[1101]: REQUEST localhost - - "POST / HTTP/1.1" 200 181 Renew-Subscription client-error-not-found
Dec 5 22:46:08 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:08 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:46:19 centos9 NetworkManager[1077]: <info> [1733431570.4927] agent-manager: agent[1dd77baa26625395,1.67/org.gnome.Shell.NetworkAgent/1000]: agent registered
Dec 5 22:46:38 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:46:37 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:37 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:47:27 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:48:37 centos9 systemd[4230]: Started VTE child process 8128 launched by gnome-terminal-server process 6210.
```

```

[MG@centos9 ~]$ ps aux | grep tail
root      8125  0.0  0.2 239788  956 pts/2    S+   22:48   0:00 sudo tail -f /var/log/messages
root      8127  0.0  0.0 229984  1792 pts/2    S+   22:48   0:00 tail -f /var/log/messages
MG        8164  0.0  0.0 221664  2304 pts/0    S+   22:48   0:00 grep --color=auto tail

[MG@centos9 ~]$ kill -15 8125
[MG@centos9 ~]$
```

```

[MG@centos9 ~]$ sudo tail -f /var/log/messages # Start a dummy process
Dec 5 22:15:18 centos9 systemd[1]: dnf-makecache.service: Deactivated successfully.
Dec 5 22:15:18 centos9 systemd[1]: Finished dnf makecache.
Dec 5 22:25:18 centos9 cupsd[1101]: REQUEST localhost - - "POST / HTTP/1.1" 200 181 Renew-Subscription client-error-not-found
Dec 5 22:46:08 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:08 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:46:19 centos9 NetworkManager[1077]: <info> [1733431570.4927] agent-manager: agent[1dd77baa26625395,1.67/org.gnome.Shell.NetworkAgent/1000]: agent registered
Dec 5 22:46:38 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:46:37 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:37 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:47:27 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:48:37 centos9 systemd[4230]: Started VTE child process 8128 launched by gnome-terminal-server process 6210.
Terminated
[MG@centos9 ~]$
```




Priority and Nice Values in Linux

Some processes may be highly CPU-intensive but not as important as others and hence can have a lower priority while others may or may not be highly CPU intensive but are very important and hence should have higher priority.

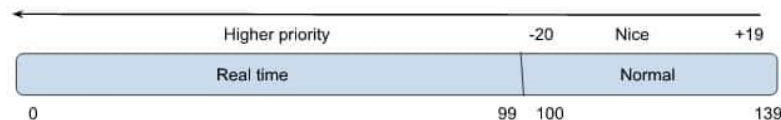
- For example, if there is a process **A**, which detects fraud with input data and there is another process **B**, which makes hourly backups of some data, then the **priority(A) > priority(B)**.
- This ensures that if both **A** and **B** are running at the same time, **A** would be allocated more processing bandwidth.

Priority Value: Determines the actual priority of a process, used by the Linux kernel for scheduling.

- **Range:** 0–139 (real-time: 0–99, user-space: 100–139).

Real-time processes in Linux are high-priority tasks that must complete within strict timing constraints, ensuring they get CPU time before other processes (e.g., embedded systems, telecommunications, ..., etc.).

User processes in Linux are regular tasks initiated by normal users, with lower priority than real-time processes, and their execution can be adjusted using the nice value.



Nice Value: A user-controlled value to influence process priority.

- **Range:** -20 to +19 (-20 is highest priority, 0 is default, +19 is lowest priority)

$$\text{Priority Value} = \text{Nice Value} + 20$$

To see how these work together let us run a process that takes a lot of processing power continuously, we will use a shell script (**infinite.sh**) which uses an infinite loop in it to demonstrate how this works.

```
$ nano infinite.sh
#!/bin/bash
while true; do echo "Running..." > /dev/null 2>&1; done
```

Run `$ sh infinite.sh &` (or `chmod u+x infinite.sh` then `./infinite.sh`) to run the script in the background, use `$ top` then `CTRL+P` to see that this process (PID-3227, for example) is taking high processing power of the CPU.



```

MG@centos9:~ — top
top - 08:29:39 up 32 min, 2 users, load average: 1.77, 0.48, 0.19
Tasks: 228 total, 4 running, 224 sleeping, 0 stopped, 0 zombie
%Cpu(s): 68.1 us, 30.9 sy, 0.0 ni, 0.0 id, 0.0 wa, 1.0 hi, 0.0 si, 0.0 st
MiB Mem : 3626.1 total, 909.5 free, 1212.0 used, 1762.0 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 2414.1 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
 2762 MG        20   0  774748  54372 40956 R   30.3   1.5   0:18.91 gnome-terminal-
 3227 MG        20   0  222524   3328  3072 R   30.0   0.1   0:16.09 sh

```

If we run two more processes of **infinite.sh** then all of them (e.g., PID-3227, 3328, 3295) get equal CPU as all have the same priority (e.g., %CPU = 16.9 for all the three).

```

MG@centos9:~ — top
top - 08:31:29 up 34 min, 2 users, load average: 3.25, 1.38, 0.55
Tasks: 232 total, 6 running, 226 sleeping, 0 stopped, 0 zombie
%Cpu(s): 61.5 us, 37.5 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.7 hi, 0.3 si, 0.0 st
MiB Mem : 3626.1 total, 893.7 free, 1227.1 used, 1762.6 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 2398.9 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
 2190 MG        20   0 3551284 312816 130216 R   16.9   8.4   1:05.91 gnome-shell
 2762 MG        20   0  775516  55140 40956 R   16.9   1.5   0:49.75 gnome-terminal-
 3227 MG        20   0  222524   3328  3072 R   16.9   0.1   0:46.94 sh
 3295 MG        20   0  222524   3328  3072 R   16.9   0.1   0:02.71 sh
 3328 MG        20   0  222524   3328  3072 R   16.9   0.1   0:01.14 sh

```

Now let us give these processes different **nice** values, there are two ways to do this:

- 1) Start a new process with a nice value using the **\$ nice** command

```
$ nice -n <nice_val> [command]
```

```
$ nice -n 10 sh infinite.sh &
```

- » When we run this we get another process (PID-8509) with the priority as 30 (Priority = 20 + 10 = 30).
- » As it has the least priority, it gets the least amount of CPU.

```

MG@centos9:~ — top
top - 23:55:59 up 4:26, 2 users, load average: 5.23, 2.94, 1.39
Tasks: 240 total, 7 running, 233 sleeping, 0 stopped, 0 zombie
%Cpu(s): 53.3 us, 36.8 sy, 8.9 ni, 0.0 id, 0.0 wa, 0.7 hi, 0.3 si, 0.0 st
MiB Mem : 3626.1 total, 1408.0 free, 1419.5 used, 1072.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 2206.6 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM    TIME+  COMMAND
 8509 MG        30  10  222520   3328  3072 R   14.3   0.1   0:49.15 sh
 8585 MG        20   0  222520   3328  3072 R   14.3   0.1   0:35.67 sh
 8626 MG        20   0  222520   3456  3200 R   14.3   0.1   0:33.12 sh
 8668 MG        20   0  222520   3328  3072 R   14.3   0.1   0:04.79 sh
 4525 MG        20   0 3648856 400624 136752 R   14.0  10.8   7:19.55 gnome-shell
1103 root        20   0   16620   9216   7936 S    0.0   0.2   0:00.02 sshd
4650 MG        20   0   58728  25036  19328 S    0.0   0.7   0:00.23 gnome-shell-cal
5092 MG        20   0   678840 15352   9600 S    0.0   0.4   0:00.05 gsd-sharing
7343 MG        20   0   224504   6016   3712 S    0.0   0.2   0:00.08 bash
8514 MG        20   0   224112   5632   3712 S    0.0   0.2   0:00.01 bash

```



2) Change a running process nice value by its **PID** using `$ renice` command

```
$ renice -n <nice_val> -p [PID]
```

```
$ renice -n 5 -p 8585
```

- » When we run this, the process with PID-8585 get its priority changed to 25 (Priority = 20 + 5 = 25).
- » The CPU is allocated accordingly as well.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
4525	MG	20	0	3648856	400516	136752	R	14.2	10.8	8:39.84	gnome-shell
8509	MG	30	10	222520	3328	3072	R	14.2	0.1	2:56.11	sh
8668	MG	20	0	222520	3328	3072	R	14.2	0.1	2:11.75	sh
8585	MG	25	5	222520	3328	3072	R	13.9	0.1	2:42.63	sh
8626	MG	20	0	222520	3456	3200	R	13.9	0.1	2:40.08	sh
1103	root	20	0	16620	9216	7936	S	0.0	0.2	0:00.02	sshd

Note that:

- » Regular users can only set positive nice values (0 to 19) to lower a process's priority.
- » Only the root user (administrator) can set negative nice values (-20 to -1) to increase a process's priority.

--- Summary ---

Feature	nice	renice
When used	When starting a new process	For changing running processes
Root privilege	Required for <u>negative nice values</u>	
Effect	Sets the initial nice value	Modifies an existing nice value